

Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет телекоммуника-
ций и информатики»

Факультет ИВТ

Кафедра вычислительных систем

Расчетно-графическая работа
на тему «РАЗРАБОТКА ИНСТРУМЕНТОВ КОМАНДНОЙ СТРОКИ ОС
GNU/LINUX»
Вариант 2.2 «Калькулятор командной строки»

Выполнил:
студент гр. ИВ-522
Синьков Г.Е.

Проверил:
старший преподаватель Кафедры ВС
Ревун А.Л.

Тема расчетно-графической работы

Разработка программного обеспечения для вычисления простейших арифметических выражений с учетом приоритета операций.

Задание на расчетно-графическую работу

Разработать программу-калькулятор `cmdcalc`, предназначенную для вычисления простейших арифметических выражений с учетом приоритета операций и расстановки скобок. На вход команды `cmdcalc` через аргументы командной строки поступает символьная строка, содержащая арифметическое выражение. Требуется проверить корректность входного выражения (правильность расстановки операндов, операций и скобок) и, если выражение корректно, вычислить его значение.

.....

Анализ задачи

Способ представления данных:

Для решения задачи выбран метод **двух стеков** (алгоритм Дейкстры), который является классическим подходом для обработки математических выражений:

- **Стек операндов (чисел):** динамический массив типа `double`. Предназначен для временного хранения чисел, извлеченных из входной строки, до момента выполнения над ними операции.
- **Стек операций:** динамический массив типа `char`. Предназначен для хранения знаков арифметических операций (+, -, *, /). Стек позволяет хранить операторы в порядке, который учитывает их приоритетность.

Основные структуры данных:

```
// Структура стека для хранения операндов (чисел) и операторов
typedef struct {
    double *nums; // Динамический массив для чисел (double)
    char *ops;    // Динамический массив для знаков операций (+, -, *, /)
    int top_num;  // Индекс вершины стека чисел
    int top_op;   // Индекс вершины стека операторов
    int size;     // Максимальный размер стека
} Stack;
```

Эта структура позволяет реализовать алгоритм вычисления арифметических выражений, разделяя поток данных на числа и операции.

Алгоритм обработки:

1. **Парсинг:** Программа проходит по строке выражения символ за символом.
2. **Разделение:**
 - Числа сразу отправляются в `num_stack` (стек чисел).
 - Операторы (+, -, *, /) проходят проверку на приоритет.
3. **Учет приоритетов:** Перед тем как положить новый оператор в `op_stack` (стек операторов), программа смотрит, что лежит на его вершине. Если там уже находится оператор с таким же или более высоким приоритетом (например, мы хотим положить +, а там *), то сначала выполняется вычисление (`process_op`), чтобы соблюсти математический порядок действий.
4. **Вычисление:** Функция `process_op` берет из `num_stack` два последних числа, применяет к ним верхний оператор из `op_stack` и результат кладет обратно в `num_stack`.
5. **Финал:** Когда строка закончилась, программа выполняет все оставшиеся в `op_stack` операции, пока он не станет пустым. Последнее число в `num_stack` и есть результат.

Тестовые данные

```
gashanus@Goshik:~/Рабочий стол/rgr_progа$ ./cmdcalc 2+2*4
Результат: 10
gashanus@Goshik:~/Рабочий стол/rgr_progа$ ./cmdcalc 10/4
Результат: 2.5
gashanus@Goshik:~/Рабочий стол/rgr_progа$ ./cmdcalc 2.5+3.14
Результат: 5.64
gashanus@Goshik:~/Рабочий стол/rgr_progа$ ./cmdcalc -5+3
Результат: -2
double free or corruption (out)
Аварийный останов (образ памяти сброшен на диск)
gashanus@Goshik:~/Рабочий стол/rgr_progа$ ./cmdcalc 5/0
Результат: inf
```

Листинг программы

Stack.h

```
1  #ifndef STACK_H
2  #define STACK_H
3
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  // Структура стека для хранения операндов (чисел) и операторов
8  typedef struct {
9      double *nums; // Динамический массив для чисел (double)
10     char *ops;     // Динамический массив для знаков операций (+, -, *, /)
11     int top_num;   // Индекс вершины стека чисел
12     int top_op;    // Индекс вершины стека операторов
13     int size;      // Максимальный размер стека
14 } Stack;
15
16 // Прототипы функций для работы со стеком
17 Stack* create_stack(int size);
18 void push_num(Stack *s, double val);
19 double pop_num(Stack *s);
20 void push_op(Stack *s, char op);
21 char pop_op(Stack *s);
22 int is_empty_op(Stack *s); // Проверка стека операторов на пустоту
23 char peek_op(Stack *s);    // Посмотреть верхний оператор без удаления
24 void free_stack(Stack *s); // Очистка памяти
25 #endif
26
```

Stack.c

```
#include "stack.h"
#include <stdlib.h>

// Создание стека: выделяем память под массивы для чисел и операторов
Stack* create_stack(int size) {
    Stack *s = (Stack*)malloc(sizeof(Stack));
    s->nums = (double*)malloc(size * sizeof(double));
```

```

    s->ops = (char*)malloc(size * sizeof(char));
    s->top_num = -1; // -1 значит, что стек пуст
    s->top_op = -1;
    s->size = size;
    return s;
}

// Добавление числа (Push)
void push_num(Stack *s, double val) { s->nums[++s->top_num] = val; }

// Извлечение числа (Pop)
double pop_num(Stack *s) { return s->nums[s->top_num--]; }

// Добавление оператора (Push)
void push_op(Stack *s, char op) { s->ops[++s->top_op] = op; }

// Извлечение оператора (Pop)
char pop_op(Stack *s) { return s->ops[s->top_op--]; }

// Просмотр верхнего оператора (Peek)
char peek_op(Stack *s) { return s->ops[s->top_op]; }

// Проверка, есть ли операторы в стеке
int is_empty_op(Stack *s) { return s->top_op == -1; }

// Освобождение динамической памяти
void free_stack(Stack *s) {
    free(s->nums);
    free(s->ops);
    free(s);
}

```

Cmdcalc.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <ctype.h>
4  #include <string.h>
5  #include "stack.h"
6  // 1. Определение приоритета операций
7  int get_precedence(char op) {
8      if (op == '+' || op == '-') return 1;
9      if (op == '*' || op == '/') return 2;
10     return 0;
11 }
12 // 2. Выполнение одной операции из стека
13 void process_op(Stack *s) {
14     double v2 = pop_num(s); // Второе число
15     double v1 = pop_num(s); // Первое число
16     char op = pop_op(s);    // Оператор
17     if (op == '+') push_num(s, v1 + v2);
18     if (op == '-') push_num(s, v1 - v2);
19     if (op == '*') push_num(s, v1 * v2);
20     if (op == '/') push_num(s, v1 / v2);
21 }
22 int main(int argc, char *argv[]) {
23     if (argc < 2) {
24         printf("Ошибка: передайте выражение в кавычках\n");
25         return 1;
26     }
27     char *expr = argv[1];
28     Stack *s = create_stack(strlen(expr)); // Динамическое выделение памяти
29     // 3. Главный цикл обработки строки

```

```
30     for (int i = 0; expr[i] != '\0'; i++) {
31         if (isspace(expr[i])) continue; // Игнорируем пробелы
32
33         if (isdigit(expr[i]) || expr[i] == '.') {
34             // Исправленное чтение числа
35             char *endptr;
36             push_num(s, strtod(&expr[i], &endptr));
37             i = endptr - expr - 1;
38         } else {
39             // Если пришел оператор, проверяем приоритеты
40             while (!is_empty_op(s) && get_precedence(peek_op(s)) >= get_prece-
41 dence(expr[i])) {
42                 process_op(s);
43             }
44             push_op(s, expr[i]);
45         }
46     }
47     // 4. Выполняем все оставшиеся операции
48     while (!is_empty_op(s)) {
49         process_op(s);
50     }
51     // Вывод итогового результата
52     printf("Результат: %g\n", pop_num(s));
53     free_stack(s); // Очистка памяти
54     return 0;
}
```